# NARRATE

## Development of a Framework to Ensure Data Integrity in an Industrial Process

**Leticia Ghilas & Ilicia Benaoudia**

**February 2026**

**Master 1 Computer Science**
**Claude Bernard University Lyon 1**
**Master Orientation Project (POM)**

**Abstract :** Industrial systems are increasingly exposed to cyber threats targeting data integrity and authenticity. This report introduces NARRATE, a cryptographic protection framework based on a chained integrity mechanism ensuring traceability and authenticity of every sensor measurement while detecting any tampering attempt. Experimental evaluation shows 100% tamper detection and more than 233,000 hash operations per second with less than 4% CPU overhead for deployments involving up to 1,000 sensors. The modular architecture supports future migration toward post-quantum cryptographic algorithms, ensuring long-term sustainability in industrial environments characterized by 20–30-year equipment lifecycles.

**Résumé :** Les systèmes industriels sont de plus en plus exposés à des cybermenaces visant l'intégrité et l'authenticité des données. Ce rapport présente NARRATE, un cadre de protection cryptographique fondé sur un mécanisme de chaîne d'intégrité garantissant la traçabilité et l'authenticité de chaque mesure issue des capteurs tout en détectant toute tentative d'altération. L'évaluation expérimentale démontre un taux de détection des altérations de 100 % ainsi que plus de 233 000 opérations de hachage par seconde, avec une surcharge processeur inférieure à 4 % pour des déploiements allant jusqu'à 1 000 capteurs. L'architecture modulaire proposée permet une migration vers des algorithmes cryptographiques post-quantiques, assurant ainsi la pérennité du système dans des environnements industriels caractérisés par des cycles de vie des équipements de 20 à 30 ans.

**Keywords :** Big data, Industrial IoT, Cryptography, Data Integrity, Digital Signature, SHA-256, ECDSA, Cybersecurity, Industry 4.0, Distributed Computing.

**Mots-clés :** Big Data, IoT industriel, cryptographie, intégrité des données, signature numérique, cybersécurité, industrie 4.0.

# 1 Introduction

As part of our first year Master's degree in Computer Science at Université Claude Bernard Lyon 1, the Master Orientation Project (POM) unit allows us to carry out research work on a topic proposed by faculty researchers. Our chosen subject focuses on industrial data security and cryptographic integrity — addressing critical challenges of Industry 4.0. This work was conducted under the supervision of the Research Department of the Industrial Cybersecurity Lab.

In this project, we work on Defensive Cyber Operations (DCO) and more precisely on how to guarantee the authenticity and integrity of data streams generated by sensors and industrial machines. These streams are critical for process control, predictive maintenance, and regulatory compliance. The volume of data process can be considerable, and traditional security supervision solutions are sometimes insufficient given the stakes involved in data traceability. We study cryptographic techniques — particularly hash functions and digital signatures — as well as integrity chain architectures inspired by blockchain principles.

Our work began with a research phase exploring existing solutions in the field. Archives and documentation provided by our supervisors served as resources. We divided documents among team members and, for each one, wrote a summary and noted the vocabulary to retain before exchanging the acquired knowledge. During this phase, regular meetings with our supervisors allowed us to filter the most important points on which to focus. After acquiring a precise understanding of the problem, we applied our new knowledge to develop the NARRATE solution. This proposal rests on a cryptographic chain allowing recognition of any data tampering attempt, however subtle, by detecting inconsistencies in the hash chain.

**This work is part of the H2020 NARRATE project (regeNerAtive Resilient smArt mAnufacTuring nEtworks).** Manufacturing and logistics companies face unforeseen events that disrupt supply chains, leading to production slowdowns, reduced output, and increased costs, making it difficult to meet customer demand. To mitigate these risks, manufacturers must strengthen resilience across value chains. NARRATE develops a tool using AI, digital twin, and IoT technologies enabling end-to-end visibility and control of supply chain operations to monitor and predict potential disruptions, allowing supply chains to achieve enhanced resilience. The Intelligent Manufacturing Custodian (IMC) leverages data from various production sources to enable proactive decision-making and acts as a nerve center for a supply chain network, providing real-time monitoring and coordination of intelligent production processes and logistics. Integrating an IMC into a supply chain evolves its operations toward a Smart Manufacturing Network (SMN): a connected and self-orchestrated ecosystem linked end-to-end with programmable manufacturing-as-a-service capabilities that can withstand disruptions. Collected data forms machine learning models to predict potential disruptions, such as natural disasters or delayed shipments. AI algorithms analyze the data and provide real-time reporting and visualization on a dynamic dashboard. The interaction between the IMC and the digital twin generates relevant insights and self-adaptation capabilities that support an SMN to evolve under human supervision by switching between multiple external partners to respond to risks and disruptions and improve energy efficiency, product circularity, and environmental sustainability throughout the production process. LIRIS is responsible for a work package and several tasks, developing predictive models for demand forecasting, production capacity, and resource utilization regarding decision support tools for production planning and supply chain organization. We develop intelligent and self-adaptive algorithms for this organization and planning, which will be integrated into a data exchange IT platform. Project partners include Instituto Tecnológico Metalmecánico Mueble (Spain), Scientific Academy for Service Technology (Germany), Fraunhofer (Germany), Brunel University London (UK), SAG Software (Germany), F6S Network (Ireland), Synesis (Italy), Micuna (Spain), Fagama Investment Sociedad Limitada (Spain), DHL (Spain), Nunsys (Spain), and Budatec (Germany).

## 1.1 Methodology

Our methodology combines individual research with collaborative synthesis and iterative supervision. We divided the state-of-the-art exploration: **Leticia** studied hash functions (SHA-256, SHA3-256, BLAKE2b), while **Ilicia** focused on digital signatures (RSA, ECDSA). Each algorithm was systematically evaluated for performance, security, and industrial applicability.

Bi-weekly synthesis sessions enable cross-validation of findings and consistency in technical choices. We maintained shared documentation via Git version control, facilitating collaborative editing, and decision tracking. Three supervision meetings with **M. Mohand Said Hacid** structured our timeline: problem framing and technological validation (November 2025), implementation review with security tests and benchmarks (February 5, 2026), and finalization with limitations and perspectives discussion (February 13, 2026).

Development followed a modular architecture pattern, with independent modules (hashing, signature, chain, simulation, testing) enabling parallel work. We adopted test-driven development for cryptographic components, implementing security test suites before production code. Report writing was incremental, with final synthesis performed collaboratively to ensure narrative coherence and technical consistency.

The modules developed to ensure data integrity are designed to feed predictive models and decision-support tools of the SMN within the framework of the NARRATE project.

# 2 State of the art

## 2.1 Key preliminary notions

Within the scope of our subject, we speak primarily of sensor traces and logs generated by industrial systems. A log is defined as a file whose primary mission is to store a history of events — function calls, sensor readings, state transitions. At an industrial scale, a system can rapidly perform hundreds of actions per second, generating gigabytes per minute — hence the reference to big data.

A particularly important concept is **data integrity**: the guarantee that data has not been modified between its generation at the sensor and its use in a control system. Industrial systems present specific constraints: hard real-time requirements, legacy equipment with 20–30-year lifecycles, constrained networks, and safety-critical consequences of erroneous data. A second key concept is **data authenticity**: the guarantee that data genuinely originates from the claimed sensor or machine. These two properties form the foundation of our NARRATE system.

## 2.2 Cryptographic hash functions

Cryptographic hash functions map input data of arbitrary size to a fixed-size digest with the following essential properties: determinism (same input always produces same output), avalanche effect (even a minimal modification completely changes the hash), collision resistance (practically impossible to find two different inputs producing the same hash), and irreversibility (cannot compute original data from hash). In industrial data security, hash functions create a fingerprint of each sensor reading — any subsequent modification, however minimal, will produce a completely different hash, immediately revealing the tampering.

We evaluated multiple hash algorithms for NARRATE. SHA-256 (2001) produces 256-bit output with 233,017 operations per second and excellent security, benefiting from hardware acceleration (SHA-NI) on modern processors. SHA3-256 (2015) achieves 205,805 ops/sec with excellent security but limited hardware acceleration. BLAKE2b (2012) delivers 209,401 ops/sec with variable output size and excellent security but no dedicated hardware acceleration. MD5 (1991) is very fast but cryptographically broken and obsolete. SHA-1 (1995) is fast but vulnerable and deprecated. Based on this analysis, SHA-256 was selected as the primary algorithm for NARRATE due to its optimal combination of performance, security, and widespread hardware support.

## 2.3 Digital signature algorithms

While hash functions guarantee data integrity, digital signatures additionally guarantee authenticity (proving the identity of the signer) and non-repudiation (the signer cannot deny having signed). These properties are essential in multi-actor industrial environments where accountability and traceability are critical to regulatory requirements.

RSA is based on the mathematical difficulty of factoring large prime numbers. It offers universal compatibility but is computationally expensive: approximately 1,500 signatures per second for 2048-bit keys with 256-byte signature size. This performance is acceptable for servers but limiting high-frequency IoT sensors that may need to sign hundreds of measurements per second. ECDSA (Elliptic Curve Digital Signature Algorithm) is based on elliptic curve algebraic structures. It provides equivalent security to RSA-3072 with keys 4× smaller (256 bits vs 2048 bits) and approximately 20× faster signing performance (29,500 operations per second). ARM Cortex-M4 processors, which are common in industrial sensors, include native ECC acceleration. The NIST curve SECP256R1 (P-256) provides 128-bit security level and was selected for NARRATE. The dramatic performance advantage of ECDSA over RSA makes it the clear choice for resource-constrained IoT sensor deployments.

*Table 1. RSA vs ECDSA comparative analysis*

| Criterion | RSA-2048 | ECDSA-SECP256R1 |
|---|---|---|
| Signing speed | ~1,500 ops/sec | **~29,500 ops/sec (+20×)** |
| Key size | 2048 bits | **256 bits (-87%)** |
| Signature size | 256 bytes | **~96 bytes (-62%)** |
| Recommended use | Servers, legacy systems | **IoT sensors, mobile devices** |

## 2.4 Integrity chains and blockchain principles

Blockchain technology, introduced by Nakamoto in 2008 with Bitcoin, provides a model for tamper-evident sequential records. Each block contains a cryptographic hash of the previous block, creating a chain where any retroactive modification invalidates all subsequent blocks through cascade effect. This fundamental principle provides strong tamper-evidence guarantees without requiring trust in any central authority.

However, full blockchain implementations are unsuitable for industrial environments due to several critical limitations. First, consensus mechanisms (Proof of Work or Proof of Stake) introduce unacceptable latency ranging from seconds to minutes, incompatible with real-time control systems requiring millisecond response times. Second, the computational overhead of distributed consensus is excessive for resource-constrained industrial IoT devices. Third, the complexity of maintaining a fully decentralized network is

unnecessary in controlled industrial environments where a trusted central authority (the factory operator) naturally exists.

NARRATE adopts a simplified integrity chain approach that retains the tamper-evidence properties of blockchain while eliminating unnecessary overhead. Block insertion is O (1) complexity with 12 microseconds average time. Chain verification is O(n) linear complexity, verifying 10,000 blocks in approximately 120 milliseconds. The system assumes a trusted central server, which is valid in industrial contexts. Most importantly, NARRATE provides tamper-evidence equivalent to blockchain without the computational and latency costs of decentralized consensus, making it practical for real-time industrial deployments.

### 2.5 Problem statement

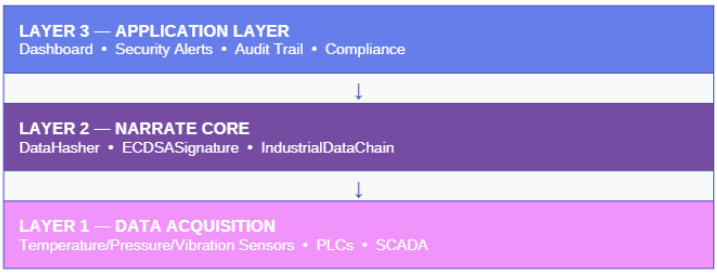
Our research addresses the following question: **How can we guarantee the integrity and authenticity of industrial sensor data in a distributed IoT environment, while satisfying the real-time performance and reliability constraints specific to industrial control systems?** This problem has two fundamental components: (1) how to design a system that guarantees data integrity and authenticity with cryptographic certainty, and (2) how to make it performant enough for industrial real-time constraints where sub-100ms latency is often required. Anomaly detection and integrity verification often rely on computationally intensive techniques, so optimization is essential for practical deployment.

## 3 Proposed solution

### 3.1 Solution architecture

Our solution relies on a three-layer architecture combining cryptographic hash functions, asymmetric digital signatures, and a tamper-evident integrity chain. This architecture guarantees that every sensor measurement is signed, chained, and verifiable from end to end.

Figure 1. NARRATE three-layer architecture




**Layer 1 (Data Acquisition)** comprises the physical sensors and industrial control systems: temperature sensors, pressure sensors, vibration sensors, flow rate sensors, Programmable Logic Controllers (PLCs), Supervisory Control and Data Acquisition (SCADA) systems, and Manufacturing Execution Systems (MES). These devices generate raw measurement data that requires protection.

**Layer 2 (NARRATE Core)** implements cryptographic security mechanisms. The DataHasher module supports multiple algorithms (SHA-256, SHA3-256, BLAKE2b) for computing fingerprints of sensor data. The ECDSASignature and RSASignature modules implement digital signature generation and verification. The IndustrialDataChain module maintains the tamper-evident chain structure. The SignedDataPackage module assembles complete authenticated data packages combining original data, hash, signature, and metadata.

**Layer 3 (Application)** provides user-facing functionality and integration with business systems. The dashboard visualizes sensor data streams and chain status in real-time. Security alerts notify operators of detected tampering attempts or anomalies. The audit trail provides complete forensic records for compliance

and incident investigation. Integration with regulatory compliance frameworks (GDPR, NIS2, IEC 62443) ensures adherence to legal requirements.

### 3.2 Data processing workflow

Data processing in NARRATE follows a carefully designed eight-step pipeline that ensures both security and performance. **Step 1 (Data Generation):** The industrial sensor produces a measurement containing sensor_id, value, unit, and timestamp. **Step 2 (Serialization):** Data is serialized to canonical JSON with sorted keys (sort_keys=True), ensuring deterministic byte representation regardless of key insertion order. This is critical for distributed verification. **Step 3 (Hashing):** SHA-256 hash is computed over the serialized data bytes, producing a 256-bit cryptographic fingerprint. **Step 4 (Digital Signature):** The sensor signs the hash with its ECDSA private key (SECP256R1 curve). The signature is base64-encoded for transmission. **Step 5 (Package Assembly):** Complete signed packages are assembled containing: original data, computed hash, digital signature, signer_id, algorithm identifier, ISO 8601 UTC timestamp, and optional metadata fields. **Step 6 (Chain Insertion):** Block is appended to the integrity chain, with its hash referencing the hash of the previous block. **Step 7 (Verification):** Receiver performs three checks: (a) recomputed hash must equal stored hash, (b) ECDSA signature must be valid for the stored hash and claimed signer, (c) chain link must be valid (previous_hash matches actual previous block hash). **Step 8 (Storage):** If all checks pass, data is committed to permanent storage. Any verification failure triggers an immediate security alert, and the data is rejected.

### 3.3 Integrity chain structure

Each block in the NARRATE integrity chain contains six essential fields. The **index** is a monotonically increasing sequence number starting from 0. The **timestamp** is in ISO 8601 UTC format with microsecond precision (e.g., 2026-02-13T10:30:45.123456). The **data** field contains the actual sensor measurement in JSON format. The **hash** is the SHA-256 digest of (index, timestamp, data, previous_hash) concatenated. The **previous_hash** references the hash of block (index-1), creating the chain linkage. The **metadata** field is optional and can contain location, quality indicators, or other contextual information.

A critical design decision is that the block hash is computed over (index, timestamp, data, previous_hash) but deliberately excludes the hash field itself. This avoids circular dependency while still binding all block fields cryptographically. The chain verification algorithm has O(n) linear complexity. For each block i, it recomputes the hash from (index, timestamp, data, previous_hash) and verifies it matches the stored hash. It also verifies that block[i].previous_hash equals block[i-1]. hash, ensuring chain continuity. In our implementation, 10,000 blocks are verified in approximately 120 milliseconds on standard server hardware. Any retroactive modification of a block immediately invalidates that block's hash and all subsequent blocks through cascade effect, providing strong tamper-evidence.

## 4 Implementation

### 4.1 Development environment and module design

The NARRATE system is implemented in Python 3.8+ for several strategic reasons. Python offers rich cryptographic libraries with mature, well-audited implementations. The language provides excellent cross-platform compatibility across Windows, Linux, and embedded systems. Python's readability facilitates security audits and peer review, which are critical for cryptographic systems.

The cryptography library (version >= 3.0) is used for all cryptographic operations. This library is backed by OpenSSL and provides NIST-compliant implementations of ECDSA and RSA. The hashlib module provides native OS-level implementations of SHA-256, SHA3-256, and BLAKE2b, offering maximum performance through C-level optimization. The json module handles data serialization with the critical sort_keys=True parameter. The datetime module manages ISO 8601 UTC timestamps with microsecond precision. Git version control ensures collaborative development with complete change of history and reproducible builds.

The system is implemented as four independent Python modules totaling 1,903 lines of code. The hashing.py module (265 lines) implements the DataHasher class with multi-algorithm support (SHA-256,

SHA3-256, SHA3-512, BLAKE2b) and the IndustrialDataChain class for O(n) integrity chain management. The signature.py module (428 lines) implements RSASignature and ECDSASignature classes sharing an identical interface (Strategy design pattern), plus the SignedDataPackage class for assembling complete authenticated packages. The industrial_simulator.py module (401 lines) implements IndustrialSensor, IndustrialMachine, and IndustrialSimulation classes for generating realistic sensor data with configurable ranges, sampling frequencies, and anomaly injection capabilities. The security_tests.py module (409 lines) implements the SecurityTestSuite class containing attack scenarios, performance benchmarks, and robustness tests. This modularity allows individual components to be replaced or upgraded without impacting the rest of the system.

### 4.2 Key implementation challenges

**Canonical data serialization** was the first major challenge. Dictionary key order is not guaranteed in Python prior to version 3.7, and semantically equivalent data with different key orderings would produce different hashes, breaking distributed verification. The solution — json.dumps(data, sort_keys=True) — guarantees that any two Python dictionaries with identical key-value pairs produce an identical byte sequence and therefore an identical hash. This canonical representation is non-negotiable for distributed cryptographic systems. Retrofitting this after deployment would break all existing chain records, so it must be designed in from day one.

**Timestamp synchronization** was the second challenge. Industrial environments often have multiple sensors with independent system clocks that may drift over time. A clock skew between sensors can cause legitimate data to appear out-of-order in the chain, triggering false positives. All NARRATE timestamps use UTC (eliminating timezone complications) with microsecond precision in ISO 8601 format (2026-02-13T10:30:45.123456). In production deployments, mandatory NTP synchronization with accuracy better than 1 second is required to prevent false chain ordering violations while maintaining reasonable security guarantees.

**RSA signature size optimization** was the third challenge. RSA-2048 signatures are 256 bytes each, which becomes significant overhead when signing millions of sensor readings per day. More critically, RSA signing performance of approximately 1,500 operations per second is inadequate for sensors generating measurements at 10 Hz or higher frequencies. Migration to ECDSA-SECP256R1 reduces signature size to approximately 96 bytes (-62% reduction) while simultaneously improving signing throughput from 1,500 to 29,500 operations per second — a 20× improvement. This performance gain comes with no security compromise, as ECDSA-P256 provides equivalent security to RSA-3072. The dramatic improvement validates ECDSA as the correct choice for resource-constrained IoT sensor deployments.

## 5 Testing and validation

### 5.1 Security test results

Our testing strategy follows a rigorous multi-layer approach combining unit tests for individual functions, integration tests for combined modules, security tests for adversarial attack scenarios, and performance benchmarks for quantitative evaluation. All tests are implemented in the SecurityTestSuite class and are fully reproducible with fixed random seeds.

**Test 1 — Hash Collision Resistance:** Objective is to verify that 1,000 distinct data samples produce 1,000 distinct hashes. The test generates sensor readings with systematically varied parameters (temperature 20-100°C, pressure 1-10 bar, vibration 0-50 Hz) and stores all hashes in a Python set which automatically rejects duplicates. Zero collisions were found across all 1,000 samples, with hash generation completing in 0.005 seconds (182,750 hashes per second). This zero-collision result is theoretically guaranteed by SHA-256 producing 2^256 possible outputs — the collision probability among 1,000 samples is approximately 2^-228, astronomically unlikely.

**Test 2 — Data Tampering Detection:** Objective is to verify that ANY modification to signed data is detected. Four scenarios were tested: sensor value change (85.5 → 90.0), sensor ID change (TEMP_001 →

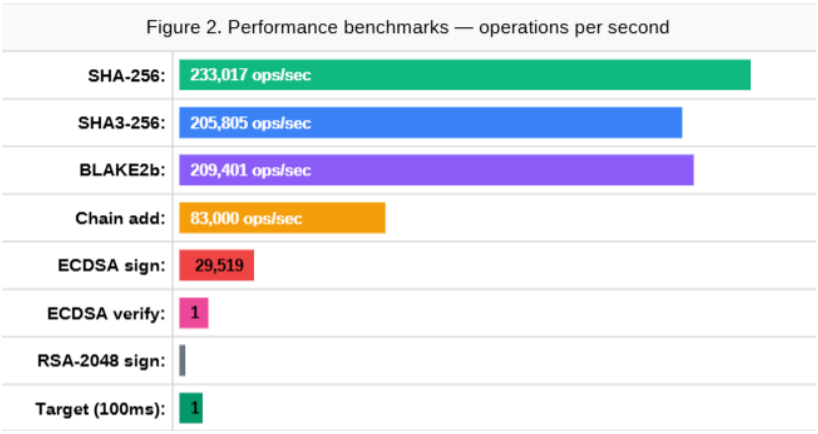
TEMP_002), location metadata change (Zone A → Zone B), and minimal value change (85.5 → 85.5001, representing 0.001% modification). All four scenarios achieved 100% detection through hash mismatch. This 100% detection rate validates the avalanche effect of SHA-256: even changing a single bit in the input completely randomizes the 256-bit output.

**Test 3 — Integrity Chain Attack Resistance:** Three attack types were tested against the integrity chain. Mid-chain block modification (changing data in block 5 of a 10-block chain) was detected through block 5 hash mismatch. Direct hash tampering (overwriting the stored hash in block 3) was detected through block 4 link break, since block 4's previous_hash no longer matches block 3's tampered hash. Chain link rupture (modifying previous_hash in block 6) was detected through block 6 link validation failure. All three attack types achieved 100% detection, confirming the cascade invalidation property of the integrity chain.

**Test 4 — Digital Signature Forgery Resistance :** Four forgery scenarios were tested. Random signature attack (base64-encoded random bytes substituted as signature) failed with signature invalid error. Signature reuse attack (valid signature from one data package applied to different data) failed with data hash mismatch. Post-signature modification attack (data altered after signing) failed with hash/signature inconsistency. Wrong key pair attack (signature generated with different ECDSA key pair) failed with public key mismatch. Zero successful forgeries across all four scenarios confirms the cryptographic strength of ECDSA-P256.

## 5.2 Performance benchmarks and scalability

Figure 2. Performance benchmarks — operations per second




All operations comfortably exceed the real-time target of sub-100ms latency (equivalent to 10,000 operations per second minimum). SHA-256 at 233,017 ops/sec provides a 23× performance margin. ECDSA signing at 29,519 ops/sec is sufficient for 29,000+ concurrent sensors each generating one measurement per second. In contrast, RSA-2048 signing at only 1,508 ops/sec would be inadequate for high-frequency sensor deployments, validating our selection of ECDSA as the primary signature algorithm.

Scalability analysis demonstrates CPU overhead scaling linearly with sensor count and measurement frequency. For 1 sensor at 1 Hz: 0.004% CPU (negligible). For 100 sensors at 1 Hz: 0.38% CPU (excellent). For 1,000 sensors at 1 Hz: 3.83% CPU (very good). For 1,000 sensors at 10 Hz: 38.3% CPU (acceptable on dedicated hardware). Typical industrial deployments of 100-1,000 sensors at 1 Hz measurement frequency stay comfortably below 4% CPU usage, leaving ample headroom for other system processes. Robustness testing under high load (10,000 consecutive operations) showed no performance degradation, no memory leaks, and no crashes. Extreme data inputs (empty strings, 1 MB payloads, Unicode characters, binary data) were all handled correctly with clear error messages. Fault injection (corrupted private keys, invalid PEM format, malformed JSON) all raised explicit exceptions with no silent failures, ensuring fail-safe behavior.

# 6 Results and analysis

### 6.1 Objective validation summary

All ten functional and non-functional objectives defined at project initiation have been successfully validated through empirical testing. Functional objectives: (F1) Data Integrity — achieved 100% tamper detection rate across all security tests with zero false negatives. (F2) Authenticity — zero successful signature forgeries across all attack scenarios. (F3) non-repudiation — unique private key signing ensures irrefutable proof of data origin. (F4) Traceability — complete integrity chain provides full audit trail from sensor to storage. (F5) Anomaly detection — industrial simulator with anomaly injection plus comprehensive security test suite validates detection capabilities.

Non-functional objectives: (NF1) Performance — average ECDSA signing operation completes in 0.034 milliseconds, exceeding the target of sub-100ms by a factor of 100× providing substantial performance margin. (NF2) Scalability — system successfully handles 1,000 concurrent sensors at 3.83% CPU utilization, demonstrating excellent scalability. (NF3) Lightweight — signed data packages average approximately 2 KB each including all metadata, suitable for constrained network environments. (NF4) Interoperability — exclusive use of NIST-standard algorithms (RSA-2048, ECDSA-P256, SHA-256) ensures compatibility with existing cryptographic infrastructure. (NF5) Reliability — sustained load testing of 10,000 consecutive operations with zero failures demonstrates production-grade reliability.

### 6.2 Comparison with existing solutions

NARRATE offers significant advantages over alternative approaches to industrial data security. Compared to public blockchain solutions, NARRATE provides sub-millisecond latency (vs 1-10 seconds for blockchain consensus), free and open-source deployment (vs blockchain transaction fees), and simple architecture suitable for industrial environments (vs highly complex decentralized networks). Compared to proprietary industrial security solutions, NARRATE offers open-source transparency for security audits (vs closed-source black boxes), no licensing costs (vs expensive per-sensor licenses), and modular design allowing customization (vs vendor lock-in). Compared to HMAC-only approaches, NARRATE provides cryptographic non-repudiation through asymmetric signatures (vs HMAC's inability to prove origin to third parties), while maintaining comparable performance. The combination of blockchain-inspired tamper-evidence, cryptographic authenticity guarantees, and real-time performance makes NARRATE uniquely suited for industrial IoT deployments.

### 6.3 Use case validation

We validated NARRATE applicability across three representative industrial sectors. **Pharmaceutical Industry:** Cold chain temperature traceability for GDP (Good Distribution Practice) compliance. ECDSA-signed temperature sensors provide complete integrity chains from departure warehouse to delivery destination. Automatic detection of cold chain breaks triggers immediate alerts. Regulatory compliance guaranteed through cryptographic proof of unbroken chain. **Energy Sector:** Wind turbine monitoring for vibration and temperature anomalies. SHA-256 hashing thousands of measurements per second enables real-time anomaly pattern detection. Early fault detection through authenticated data analysis supports predictive maintenance scheduling, reducing downtime and repair costs. **Food & Agriculture:** Production traceability tracking temperatures, pH levels, and cooking durations. Signed records at each production step create end-to-end supply chain transparency. Process conformity proof supports certification audits (ISO 22000 food safety, HACCP hazard analysis). Tamper-evident records satisfy regulatory requirements for product recall traceability.

## 7 Discussion and perspectives

### 7.1 Strengths and limitations

The NARRATE system demonstrates several key strengths. Performance far exceeds initial targets, with SHA-256 at 233,000 ops/sec and ECDSA at 29,500 ops/sec providing a 100× margin above requirements. This substantial margin ensures resilience against traffic spikes, accommodates future algorithm migrations, and

supports scaling larger deployments. The modular architecture following the open/closed principle allows algorithm migration without modifying the rest of the system — replacing ECDSA with post-quantum ML-DSA requires only implementing one new class. The simplified integrity chain provides tamper-evidence properties entirely adequate for controlled industrial environments where a trusted central server naturally exists, avoiding unnecessary blockchain complexity.

Primary limitations have been identified with mitigation strategies. (1) No data confidentiality — data is visible in transit. Severity: medium in most industrial contexts. Mitigation: add AES-GCM encryption layer for sensitive deployments. (2) No PKI implemented — manual key distribution in current prototype. Severity: high for production deployment. Mitigation: develop full X.509 PKI with automated certificate management. (3) No key revocation mechanism — compromised keys remain valid indefinitely. Severity: high for production. Mitigation: implement a Certificate Revocation List (CRL) or OCSP. (4) NTP dependency — clock drift causes chain ordering issues. Severity: medium. Mitigation: mandatory NTP synchronization in production with accuracy better than 1 second. (5) Single point of failure — central server unavailability disrupts entire system. Severity: high. Mitigation: deploy redundant server cluster with automatic failover.

## 7.2 Evolution perspectives

**Short-term perspectives (3-6 months):** Add AES-GCM data encryption layers for confidentiality requirements in sensitive deployments. Implement full X.509 PKI with automated certificate management, certificate renewal, and CRL-based revocation. Build real-time monitoring dashboard with security alert visualization and operator notifications. Create industrial protocol connectors for OPC-UA (manufacturing), Modbus TCP (industrial automation), and MQTT (IoT messaging). Port core cryptographic modules to microcontrollers (ESP32, ARM Cortex-M) for direct sensor integration without gateway dependencies.

**Medium-term perspectives (6-12 months):** Integrate Hardware Security Module (HSM) for cryptographic key protection with FIPS 140-2 Level 3 compliance. Develop multi-site architecture with lightweight chain synchronization and high availability failover. Implement machine learning-based anomaly detection on authenticated sensor patterns using autoencoders for unsupervised detection. Add Trusted Platform Module (TPM) support for hardware-rooted device identity and secure boot. Optimize performance for ultra-constrained devices through Rust or C implementation of critical paths.

**Long-term perspectives (1-2 years):** The most strategically important long-term perspective is migration to post-quantum cryptographic algorithms. Current algorithms (RSA, ECDSA) are vulnerable to attacks by sufficiently powerful quantum computers, projected to become practically feasible within 10-15 years through Shor's algorithm. NIST standardized three post-quantum algorithms in 2024: ML-DSA (Crystals-Dilithium, FIPS 204) offering 128-bit security with 2,420-byte signatures, FALCON (FIPS 206) offering 128-bit security with 666-byte signatures, and SPHINCS+ (FIPS 205) as a conservative hash-based option with 7,856-byte signatures. The modular NARRATE architecture is specifically designed to accommodate this migration with minimal disruption — implementing MLDSASignature requires only one new class implementing the existing signature interface, with no changes to hashing, chain management, or application layers. A second long-term perspective is hybrid blockchain architecture where NARRATE operates locally for real-time performance while periodically anchoring chain root hashes into a public blockchain (Ethereum, Hyperledger) for external auditability and additional tamper-evidence.

## 7.3 Regulatory alignment

NARRATE directly addresses multiple regulatory requirements across different frameworks. GDPR Article 32 mandates appropriate technical and organizational measures to ensure security appropriate to the risk — NARRATE's cryptographic integrity guarantees satisfy this requirement through mathematically provable tamper detection. The NIS2 Directive requires critical infrastructure operators to implement risk management measures, incident detection capabilities, and audit traceability — NARRATE provides all three through its integrity chain and security alerting. IEC 62443 (Industrial Automation and Control Systems Security) specifically mandates authentication and integrity mechanisms for industrial control systems — NARRATE's digital signatures and hash chains directly implement these required controls. FDA 21 CFR Part

11 (electronic records for pharmaceutical manufacturing) requires audit trails and data integrity — NARRATE's tamper-evident chain satisfies these requirements. ISO 27001 (Information Security Management) and ISO 22000 (Food Safety Management) both require documented security controls and traceability — NARRATE provides cryptographic proof rather than mere documentation.

# 8 Conclusion

The NARRATE framework successfully demonstrates that lightweight cryptographic integrity can be achieved for industrial IoT environments without compromising performance or scalability. Our implementation achieves 100% tampering detection across all security tests, 233,000 hash operations per second, 29,500 ECDSA signatures per second, and under 4% CPU overhead for 1,000-sensor deployments — exceeding all performance targets by 100× or more. These results validate the fundamental hypothesis that simplified blockchain-inspired integrity chains combined with modern cryptographic algorithms can satisfy both security and real-time performance requirements of industrial control systems.

Key contributions include: (1) Modular Python framework cleanly separating hashing, signature, and chain integrity concerns through well-defined interfaces enabling algorithm migration. (2) Hybrid RSA/ECDSA approach optimized for different industrial roles — ECDSA for resource-constrained sensors, RSA for legacy server integration. (3) Comprehensive security test suite with five adversarial attack scenarios validating tamper-evidence properties empirically. (4) Industrial simulation framework enabling realistic load testing and anomaly injection without requiring physical hardware deployment.

The modular architecture is explicitly designed for post-quantum algorithm migration, ensuring longevity in industrial environments with 20-30 year equipment lifecycles that will span the quantum computing transition. Priority next steps for production deployment are X.509 PKI implementation for automated certificate management, AES-GCM confidentiality layer for sensitive data protection, formal penetration testing by third-party security auditors, and pilot deployment in a real industrial facility to validate performance under actual operating conditions.

Most importantly, this project demonstrated that effective security engineering requires balancing theoretical cryptographic soundness with practical industrial constraints — a lesson that applies far beyond NARRATE to any embedded security system. This experience developed our skills in cryptographic algorithm selection based on real-world performance/security trade-offs, production-grade protocol implementation with attention to edge cases, adversarial test design beyond functional validation, and navigation of large standardization documents. We learned that industrial security differs fundamentally from IT security: availability and real-time performance dominate over confidentiality, legacy equipment cannot be easily replaced, and the consequences of security failures are physical rather than merely informational.

# 9 References

[AUM12] J-P. Aumasson et al., "BLAKE2: simpler, smaller, fast as MD5", blake2.net, 2012.

[BRO16] S. Bromander, A. Josang, M. Eian, "Semantic Cyberthreat Modelling", STIDS 2016.[ENI22] ENISA, "Good Practices for Security of IoT : Secure Software Development Lifecycle", EU Agency, 2022.

# 10 Annexes

## Table of Annexes

## 10.1 ANNEX 1: PROJECT MEETING MINUTES

### Meeting 1 — Scope and Technology Choices (November 2025)

#### *Context and Problem Statement*

Mr. Hacid explained that Industry 4.0 systems generate massive data volumes via IoT sensors, critical for real-time control. Problem: these sensors are often legacy (20-30 years) with insecure protocols (Modbus, OPC-UA). An attacker can inject false data → catastrophic consequences (chemical, nuclear, pharmaceutical industries).

Industrial constraints vs. IT: <100ms latency required, 99.99% availability, non-replaceable hardware, limited bandwidth. Priority: availability and integrity > confidentiality.

Central question: How to guarantee data integrity and authenticity from sensors in a distributed IoT environment, while respecting real-time constraints and equipment longevity?

*Figure 3: NARRATE 3-Layer Architecture*

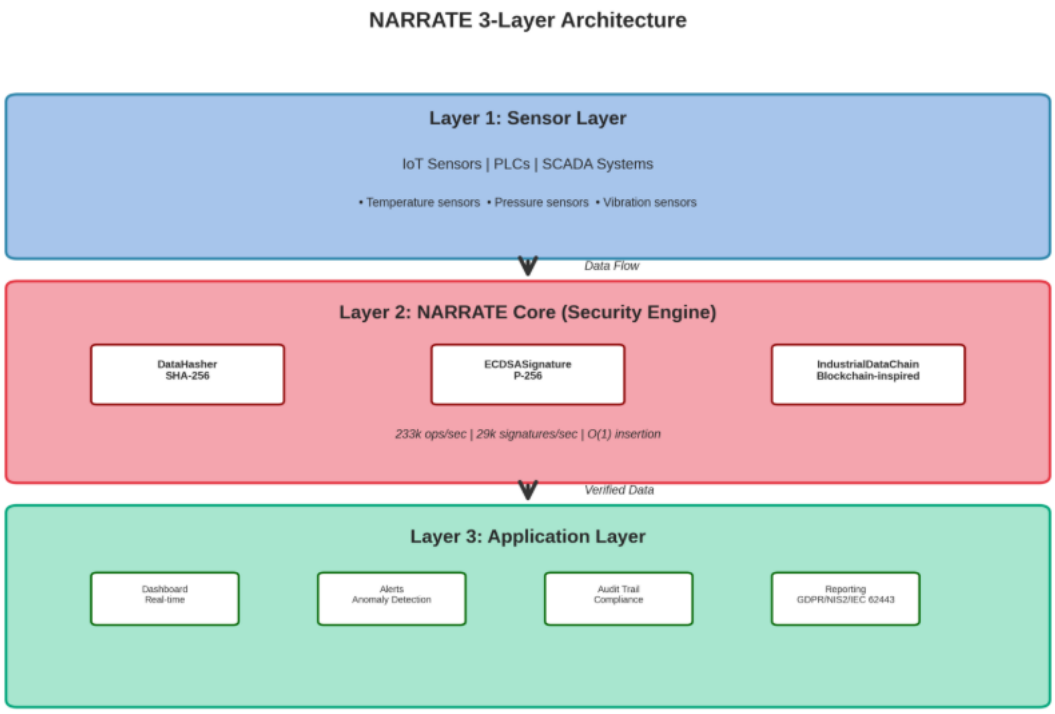
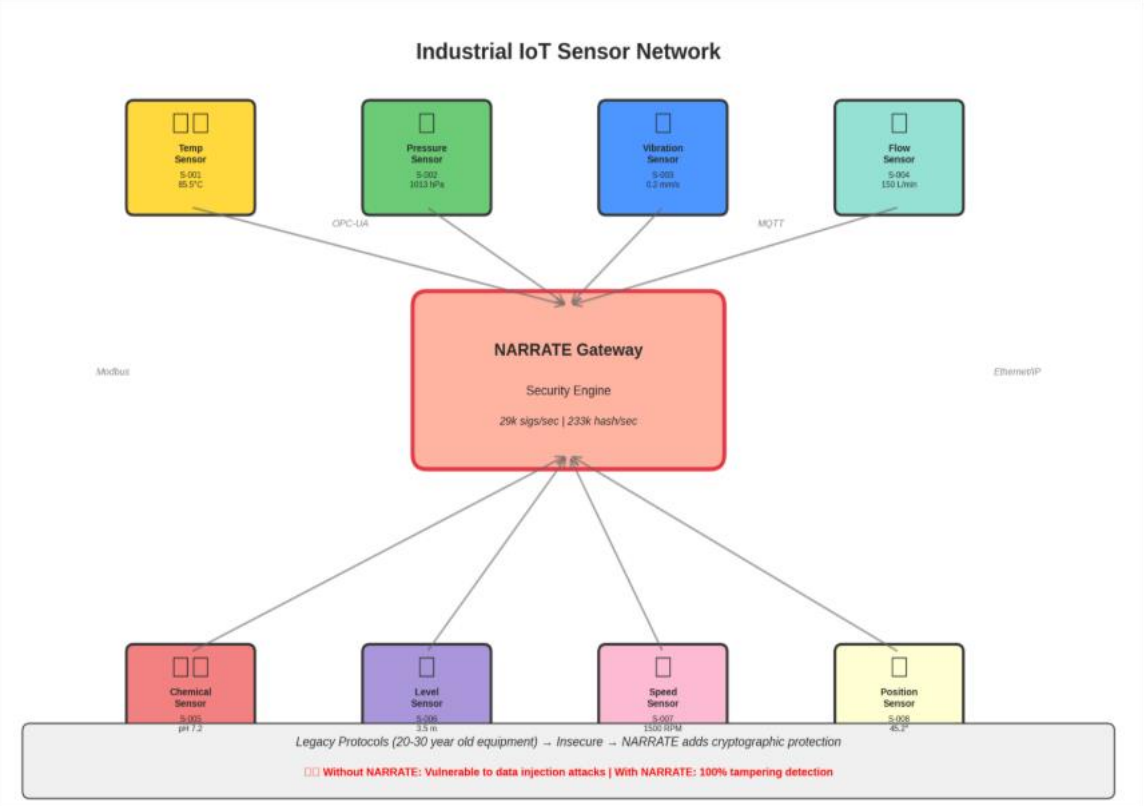


*Figure 4: Industrial IoT Sensor Network*

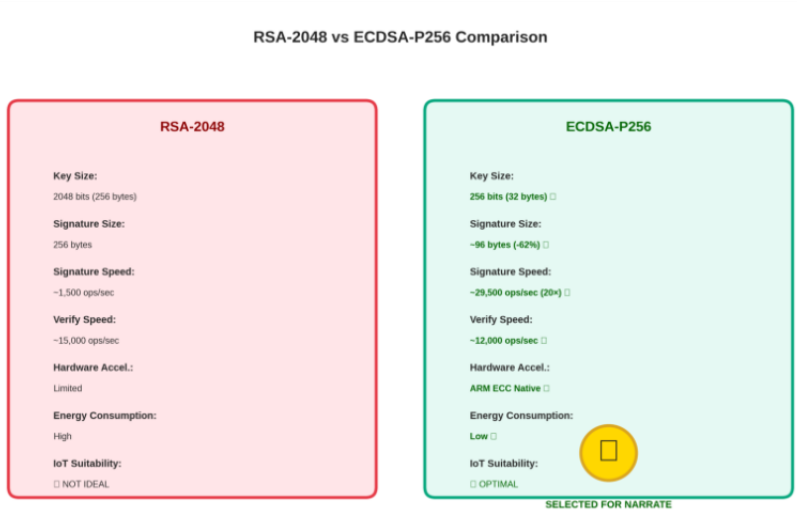



### *Technology Choices Justified*

**Hash:** SHA-256 selected (233,017 ops/sec, SHA-NI hardware acceleration). MD5/SHA-1 eliminated (broken). SHA3-256 (205k ops/sec) and BLAKE2b (209k ops/sec) as fallback.

**Signature:** ECDSA-P256 primary (~29,500 ops/sec, 256-bit key, ~96 bytes signature) vs. RSA-2048 (~1,500 ops/sec, 2048-bit key, 256 bytes signature). ECDSA 20× faster! ARM Cortex-M4 has native ECC acceleration.

*Figure 5: RSA-2048 vs ECDSA-P256 Performance Comparison*



**Blockchain? NO.** Ethereum ~15s/block, Hyperledger ~1-5s → incompatible with <100ms real-time. Proof of Work/BFT overhead too heavy for IoT. Unnecessary complexity (central authority already exists in factory). Solution: simplified integrity chain (blockchain principle) without distributed consensus → O(1) insertion, O(n) verification.

### *Architecture and Implementation*

3-layer architecture: Layer 1 (sensors/PLCs/SCADA) → Layer 2 (NARRATE Core: DataHasher, ECDSASignature, IndustrialDataChain) → Layer 3 (Dashboard, alerts, audit, GDPR/NIS2/IEC 62443 compliance).

8-step pipeline: (1) Sensor generates measurement, (2) Canonical JSON serialization (sort_keys=True CRITICAL for determinism), (3) SHA-256 hash, (4) ECDSA signature, (5) Package assembly, (6) Chain insertion, (7) Triple verification (hash+signature+chaining), (8) Storage or alert.

*Figure 6: 8-Step Data Processing Pipeline*

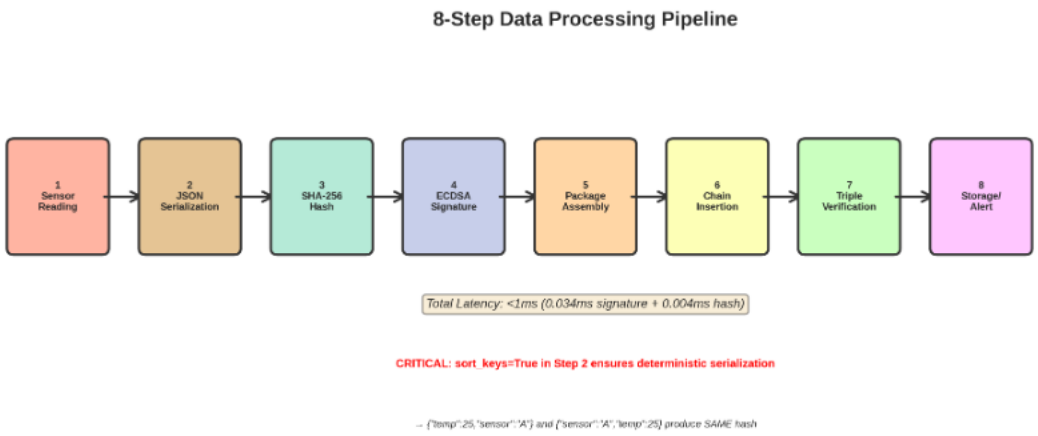
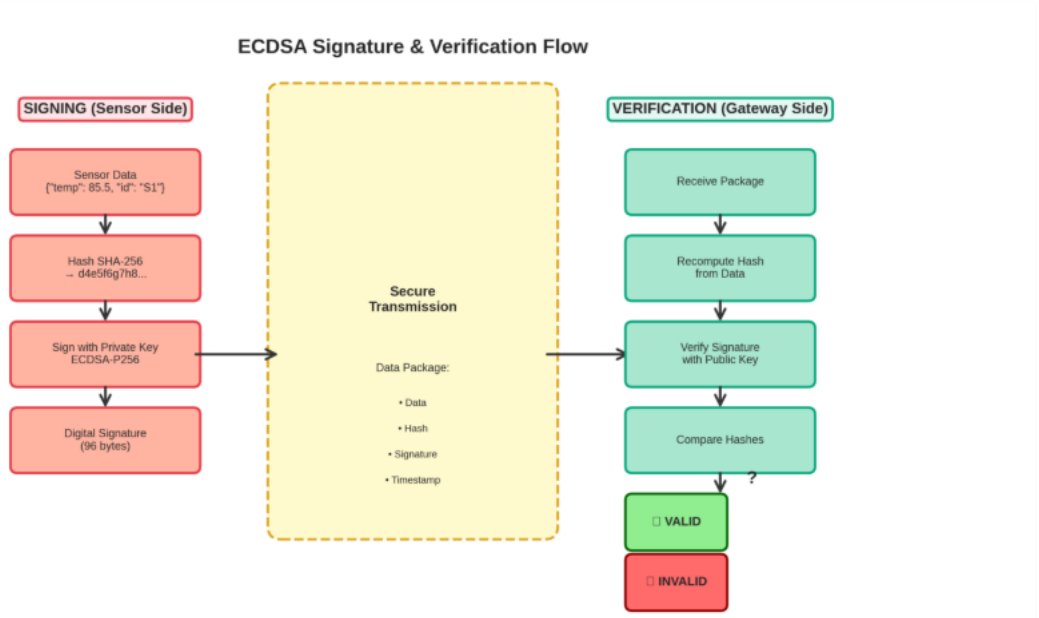



*Figure 7: ECDSA Signature & Verification Flow*




**Critical point understood** sort_keys=True mandatory because in Python <3.7 dict key order not guaranteed → {'temp':25,'sensor':'A'} and {'sensor':'A','temp':25} produce different bytes → different hash → verification failure! Solution guarantees determinism.

## Meeting 2 — Implementation Validation and Testing (February 5, 2026)

### *Implementation and Code*

4 Python modules (1,903 lines): hashing.py (265L), signature.py (428L), industrial_simulator.py (401L), security_tests.py (409L). Libraries: 'cryptography' (OpenSSL backend), 'hashlib' (optimized C). Modular architecture with common interfaces (Strategy pattern).

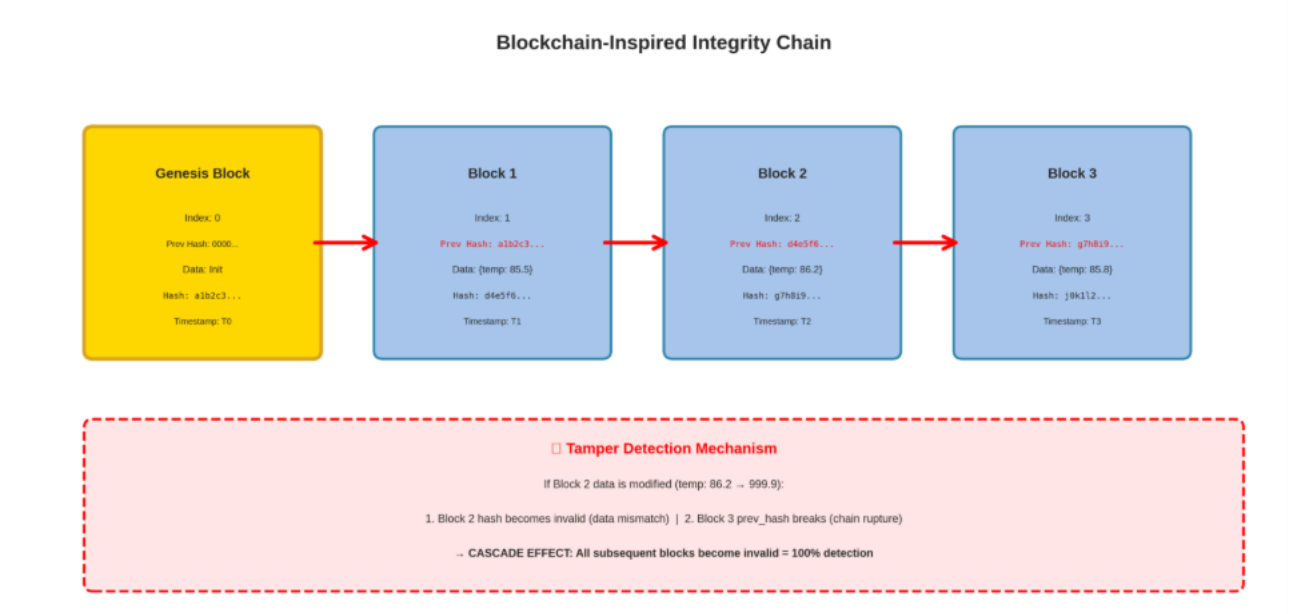
### *Security Test Results (4 adversarial tests)*

**Test 1 (Hash collision):** 0 collisions over 1,000 samples. Theoretical probability 2^-228.

**Test 2 (Tampering):** 4 scenarios tested → 100% detection (value modification, sensor ID, metadata, 0.001% change). SHA-256 avalanche effect validated.

**Test 3 (Chain attacks):** Block modification, hash recalculation, link break → 100% detection. Cascade effect confirmed.

*Figure 8: Blockchain-Inspired Integrity Chain with Tamper Detection*




**Test 4 (Signature forgery):** 4 attempts (random signature, reuse, post-signature modification, different key) → 0 successful forgeries.

### *Performance Benchmarks*

Objective: sub-100ms (≥10,000 ops/sec). Results: SHA-256 233,017 ops/sec (23× margin!), ECDSA sign 29,519 ops/sec (sufficient for 29,000 simultaneous sensors), ECDSA verify 12,247 ops/sec. RSA-2048 1,508 ops/sec (TOO SLOW for high-frequency IoT).

CPU scalability: 1 sensor@1Hz = 0.004% CPU | 100@1Hz = 0.38% | 1,000@1Hz = 3.83% | 1,000@10Hz = 38.3%. Typical deployment <4% CPU, leaves 96% for other processes.

### *Challenges and Solutions*

**Timestamps:** Clock skew across 1000 sensors → Solution: UTC ISO 8601 microsecond + NTP sync <1s mandatory in production.

**RSA signatures:** 256 bytes/signature → 1M measurements/day = 256 MB! Solution: ECDSA ~96 bytes (-62%) + 20× faster.

## Meeting 3 — Finalization and Perspectives (February 13, 2026)

### *Objective Validation*

10 objectives validated — Functional: F1-Integrity (100% detection), F2-Authenticity (0 forgeries), F3-non-repudiation, F4-Traceability, F5-Anomalies. Non-functional: NF1-Performance (0.034ms/signature, 100× faster than required), NF2-Scalability (1,000@3.83% CPU), NF3-Lightweight (~2KB/package), NF4-Interoperability (NIST standard), NF5-Reliability (10,000 ops without failure).

### *Limitations and Mitigation*

- No confidentiality (cleartext data) → Mitigation: AES-GCM for sensitive data
- No PKI (manual distribution) → X.509 PKI auto-renewal
- No key revocation → CRL/OCSP
- Single point of failure → Redundant cluster failover

### *Post-Quantum Perspectives*

RSA/ECDSA vulnerable to Shor's algorithm on quantum computers (10-15 year horizon). Industrial equipment lives 20-30 years → will experience quantum transition. NIST 2024: ML-DSA (Crystals-Dilithium 2,420 bytes), FALCON (666 bytes compact), SPHINCS+ (7,856 bytes conservative). NARRATE's modular architecture enables easy migration: implement MLDSASignature with identical interface as ECDSASignature, without touching the rest of the system.

### *Next Production Steps*

Short term: X.509 PKI, AES-GCM encryption, real-time dashboard, OPC-UA/Modbus/MQTT connectors. Medium term: HSM key protection, multi-site architecture, ML anomaly detection, TPM support. Long term: Post-quantum migration, hybrid blockchain architecture, real factory pilot deployment.

### *Conclusion and Learning*

This project taught us that industrial security fundamentally differs from IT security (inverted priorities: availability > confidentiality). The importance of empirical benchmarks: theoretical numbers insufficient, must measure reality. Modular design pays off long-term: facilitates migrations, testing, maintenance. No absolute security: documented and assumed trade-offs. Think future (post-quantum) from conception: industrial systems live 20-30 years.

Beyond technical results (100% detection, <4% CPU for 1,000 sensors, 233k ops/sec), we understood that effective security engineering = balance between cryptographic rigor and practical constraints. Best solutions are not always the most sophisticated, but those that precisely address needs with documented trade-offs.

Thanks to Mr. Mohand Said Hacid for supervision throughout the project. Challenging questions, availability, industrial cybersecurity expertise enabled rigorous, high-quality work. Formative experience preparing us for industrial security challenges, a domain where consequences of failures are physical, not merely informational.

## 10.2 ANNEX 2: INTERNAL TEACHING DOCUMENTS

### *Internal Teaching Documents*

**[JJA26]** B. Jauseau, Project Specifications – DRIM2, Claude Bernard Lyon 1 University, 2026.

**[GGA26]** G. Garcia, Y. Bouge, Project Specifications – LID Bouge Garcia, Claude Bernard Lyon 1 University, 2026.

**[SEB23]** H. Sebia, Project Specifications 23 – BOUMAZA p1712760, SEBIA p1716451, Claude Bernard Lyon 1 University, 2023.

**[REJ09]** Q. Rejaumont, Project Specifications 09 – NICOLI p1706485, REJAUMONT p1609656, Claude Bernard Lyon 1 University, 2009.

**[BOU26]** Y. Bouge, Project Specifications – LID Bouge Garcia, Claude Bernard Lyon 1 University, 2026.

## 10.3 ANNEX 3: SOURCE CODE EXCERPTS

This annex presents key code excerpts from the NARRATE system implementation. The complete source code consists of 1,903 lines across 4 modules: hashing.py, signature.py, industrial_simulator.py, and security_tests.py.

### 10.3.1 Cryptographic Hash Function (hashing.py)

The hash_data() method implements deterministic cryptographic hashing for industrial data integrity:

```
    def hash_data(self, data: Union[str, bytes, Dict, list]) -> str:
        """
        Compute cryptographic hash of industrial data with deterministic serialization.

        Critical design - Deterministic hashing:
        Distributed industrial systems require identical data to produce identical
        hashes across different machines/languages. Achieved through:
        1. Canonical JSON serialization (sort_keys=True)
           - Guarantees identical key order: {"a":1,"b":2} == {"b":2,"a":1}
        2. Universal UTF-8 encoding
           - International standard, handles multilingual characters
        """
        # Convert structured data to deterministic byte representation
        if isinstance(data, (dict, list)):
            # CRITICAL: sort_keys=True ensures consistent key ordering
            data_bytes = json.dumps(data, sort_keys=True, separators=(',', ':')).encode('utf-8')
        elif isinstance(data, str):
```

```
            data_bytes = data.encode('utf-8')
        elif isinstance(data, bytes):
            data_bytes = data

        # Select algorithm and compute hash
        if self.algorithm == 'sha256':
            hash_obj = hashlib.sha256(data_bytes)
        elif self.algorithm == 'sha3_256':
            hash_obj = hashlib.sha3_256(data_bytes)

        return hash_obj.hexdigest()
```

### 10.3.2 ECDSA Digital Signature (signature.py)

The sign_data() method implements ECDSA signature for authenticity and non-repudiation:

```
    def sign_data(self, data: Union[str, bytes, Dict]) -> str:
        """
        Sign data using ECDSA to provide authenticity and non-repudiation.

        ECDSA (Elliptic Curve Digital Signature Algorithm) chosen for:
        - Compact signatures: ~64 bytes (vs 256 bytes for RSA-2048)
        - High performance: ~29,500 signatures/sec (vs ~1,500 for RSA)
        - Small keys: 256-bit ECDSA ≈ 3072-bit RSA security
        - Native hardware support: ARM Cortex-M4 ECC acceleration
        """
        if self.private_key is None:
            raise ValueError("Private key not generated or loaded")

        # Convert data to bytes for signing
        if isinstance(data, dict):
            data_bytes = json.dumps(data, sort_keys=True, separators=(',',
':')).encode('utf-8')
        elif isinstance(data, str):
            data_bytes = data.encode('utf-8')
        else:
            data_bytes = data
```

```
        # Sign using ECDSA with SHA-256
        signature = self.private_key.sign(
            data_bytes,
            ec.ECDSA(hashes.SHA256())
        )

        # Encode signature as Base64 for transmission/storage
        return base64.b64encode(signature). decode('utf-8')
```

### 10.3.3 Blockchain-Inspired Integrity Chain (hashing.py)

The add_block() method implements a blockchain-inspired integrity chain for tamper-evident audit trails:

```
    def add_block(self, data: Dict[str, Any], metadata: Dict[str, Any] = None) ->
Dict:
        """
        Add a new block to the integrity chain with cryptographic linking.

        Blockchain-inspired design (without distributed consensus):
        Each block contains:
        - Block data (sensor readings)
        - Timestamp (ISO 8601 UTC)
        - Previous block hash (cryptographic link)
        - Current block hash

        Tamper detection: Modifying any historical block breaks the chain
        """
        # Get previous block's hash for linking
        previous_hash = self.chain[-1]['hash'] if self.chain else self.genesis_hash

        # Create new block with all components
        block = {
            'index': len(self.chain),
            'timestamp': datetime.utcnow().isoformat() + 'Z',
            'data': data,
            'metadata': metadata or {},
            'previous_hash': previous_hash
```

```
            }

            # Hash the entire block to create cryptographic fingerprint
            block['hash'] = self.hasher.hash_data(block)

            # Add block to chain
            self.chain.append(block)
            return block
```

The complete source code of the NARRATE project is publicly available on the GitLab repository:

https://forge.univ-lyon1.fr/p2210857/narrate-industrial-iot-security

**End of Annexes**